

Universidad Nacional de Río Cuarto
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación

Bases de Datos II

Examen Final

2.^{do} llamado — Turno Diciembre 2016
Resolución de estudio

Alumno: Tomás Rodeghiero
Año de estudio: 2026

Índice

1. Introducción	2
2. Pregunta 1: Protocolo de ordenamiento por marcas temporales	2
3. Pregunta 2: Pasos en el procesamiento de una consulta SQL	3
4. Pregunta 3: Información estadística para estimación de costos	5
5. Pregunta 4: Algoritmo de Merge Join	7
6. Pregunta 5: Diagrama jerárquico Open GIS	9
Cierre	10

1. Introducción

El examen consta de cinco preguntas teóricas que cubren cuatro temas centrales del cuatrimestre: control de concurrencia (Pregunta 1), procesamiento y optimización de consultas (Preguntas 2, 3 y 4) y bases de datos espaciales (Pregunta 5). Las cinco se responden en formato *Idea* → *Respuesta* → *Ejemplo*, manteniendo la convención de los prácticos. Cada pregunta apunta a una habilidad específica:

- Pregunta 1 → Teórico 8: protocolos pesimistas de concurrencia.
- Pregunta 2 → Teórico 6: pipeline de procesamiento de un motor.
- Pregunta 3 → Teórico 7: estimación de costos para el optimizador.
- Pregunta 4 → Teórico 6: algoritmos físicos de join.
- Pregunta 5 → Anexo Open GIS: jerarquía de tipos espaciales.

2. Pregunta 1: Protocolo de ordenamiento por marcas temporales

Consigna

Explique el protocolo de ordenamiento por marcas temporales. Ejemplifique.

Idea

El protocolo de *marcas temporales* (*timestamp ordering*) es un mecanismo **pesimista, no basado en bloqueos** para garantizar *serializabilidad por conflicto*. Cada transacción recibe al ingresar al sistema una marca temporal $MT(T_i)$ única y monótonamente creciente (típicamente el reloj del sistema o un contador global). El protocolo garantiza que la planificación resultante sea *equivalente al orden serial de las marcas*: si $MT(T_i) < MT(T_j)$, todo conflicto entre T_i y T_j se resuelve como si T_i se hubiera ejecutado antes que T_j . A diferencia de 2PL, no produce deadlocks (al no usar bloqueos), pero sí puede generar *rollbacks en cascada* y *starvation*.

Respuesta

Estructuras que mantiene el protocolo. Cada ítem Q guarda dos marcas:

- $MT-E(Q)$: la mayor marca temporal de las transacciones que escribieron Q con éxito.
- $MT-L(Q)$: la mayor marca temporal de las transacciones que leyeron Q con éxito.

Reglas de validación.

Cuando T_i ejecuta $r(Q)$:

- Si $MT(T_i) < MT-E(Q) \Rightarrow$ rechazo. T_i querría leer una versión de Q que ya fue sobrescrita por una transacción más nueva; T_i se retrocede y reinicia con una marca *nueva* (más grande).
- En otro caso $\Rightarrow$ se ejecuta y se actualiza $MT-L(Q) \leftarrow \max(MT-L(Q), MT(T_i))$.

Cuando T_i ejecuta $w(Q)$:

- Si $MT(T_i) < MT-L(Q) \Rightarrow$ rechazo. El valor que produciría T_i ya fue necesario por una lectura posterior; T_i se retrocede.
- Si $MT(T_i) < MT-E(Q) \Rightarrow$ rechazo. T_i querría escribir una versión obsoleta (ya hay un valor escrito por una transacción más nueva); T_i se retrocede.
- En otro caso $\Rightarrow$ se ejecuta y se actualiza $MT-E(Q) \leftarrow MT(T_i)$.

Propiedades.

- Garantiza serializabilidad por conflicto: el orden serial equivalente es el de las marcas.
- Libre de deadlocks: no hay esperas, todo conflicto se resuelve con rollback inmediato.
- Sufre *rollbacks en cascada* si una T_i que abortó ya hizo escrituras que otra leyó (a menos que se combine con la *Thomas's write rule* o con escritura diferida hasta commit).
- Posible *starvation*: una transacción larga puede ser abortada repetidamente si sus operaciones llegan tarde respecto del orden global.

Ejemplo

Sean T_1, T_2, T_3 con $MT(T_1) = 1, MT(T_2) = 2, MT(T_3) = 3$.

- T_1 : leer(X); escribir(X); leer(Y); escribir(Y);
- T_2 : leer(X); escribir(X);
- T_3 : leer(Y); escribir(Y);

Estado inicial: $MT-E(X) = MT-L(X) = 0, MT-E(Y) = MT-L(Y) = 0$.

Schedule propuesto: $r_1(X), w_1(X), r_2(X), w_2(X), r_1(Y), w_1(Y), r_3(Y), w_3(Y)$.

Paso	Operación	Validación	$MT-E(X)$	$MT-L(X)$	$MT-E(Y)$	$MT-L(Y)$
0	inicio	—	0	0	0	0
1	$r_1(X)$	$1 \geq MT-E(X) = 0$ OK; $MT-L(X) = 1$	0	1	0	0
2	$w_1(X)$	$1 \geq 1$ y $1 \geq 0$ OK; $MT-E(X) = 1$	1	1	0	0
3	$r_2(X)$	$2 \geq 1$ OK; $MT-L(X) = 2$	1	2	0	0
4	$w_2(X)$	$2 \geq 2$ y $2 \geq 1$ OK; $MT-E(X) = 2$	2	2	0	0
5	$r_1(Y)$	$1 \geq MT-E(Y) = 0$ OK; $MT-L(Y) = 1$	2	2	0	1
6	$w_1(Y)$	$1 \geq 1$ y $1 \geq 0$ OK; $MT-E(Y) = 1$	2	2	1	1
7	$r_3(Y)$	$3 \geq 1$ OK; $MT-L(Y) = 3$	2	2	1	3
8	$w_3(Y)$	$3 \geq 3$ y $3 \geq 1$ OK; $MT-E(Y) = 3$	2	2	3	3

Resultado: sin rollbacks; orden serial equivalente T_1, T_2, T_3 .

Caso con rollback. Si en cambio el schedule fuera $r_2(X), w_2(X), w_1(X)$: en el paso 3 se evaluaría $MT(T_1) = 1 < MT-E(X) = 2 \Rightarrow$ **rechazo**: T_1 se retrocede y reinicia con una marca nueva (por ejemplo $MT(T_1) = 4$).

3. Pregunta 2: Pasos en el procesamiento de una consulta SQL**Consigna**

Explique los diferentes pasos en el procesamiento de una consulta SQL por parte de un motor de bases de datos. Dé un ejemplo.

Idea

Una consulta SQL no se ejecuta tal como llega del cliente: el motor la atraviesa por un pipeline de cinco etapas que convierten progresivamente texto $\rightarrow$ árbol sintáctico $\rightarrow$ árbol algebraico $\rightarrow$ plan físico $\rightarrow$ ejecución. Las primeras etapas son determinísticas (parsing, validación); las del medio (optimización lógica y física) son donde el motor elige entre planes equivalentes basándose en estadísticas; la última (ejecución) llama al storage engine para mover bloques de disco a memoria.

Respuesta

Pipeline canónico del procesamiento de un SELECT:

1. **Análisis léxico y sintáctico (parsing).** El parser convierte la cadena SQL en una serie de tokens y los organiza en un *árbol sintáctico* (*parse tree*). Detecta errores de sintaxis (palabras reservadas mal escritas, paréntesis sin cerrar, etc.).
2. **Análisis semántico (validación).** Con el árbol sintáctico, el motor consulta el *catálogo* (`pg_catalog/INFORMATION_SCHEMA`) para validar: que las tablas y columnas referenciadas existan, que los tipos sean compatibles en los predicados, que el usuario tenga permisos. Resuelve identificadores ambiguos (a qué tabla pertenece cada columna).
3. **Traducción a álgebra relacional.** El árbol sintáctico validado se transforma en una expresión del álgebra relacional, conocida como *árbol de consulta inicial*. Cada cláusula SQL se mapea a uno o más operadores: `FROM` $\rightarrow$ producto cartesiano o join; `WHERE` $\rightarrow \sigma$; `SELECT` $\rightarrow \Pi$; `GROUP BY` $\rightarrow \gamma$; `ORDER BY` $\rightarrow$ sort.
4. **Optimización.**
 - *Lógica/heurística:* aplica reglas de equivalencia (push-down de selecciones y proyecciones, reordenamiento de joins, eliminación de joins triviales). Produce un *árbol algebraico optimizado*.
 - *Física (basada en costos):* para cada operador del árbol elige un algoritmo concreto (índice vs scan secuencial, nested loop vs hash join vs merge join, etc.) usando estadísticas del catálogo. Calcula el costo estimado de cada candidato y elige el más barato. El resultado es el *plan de ejecución*.
5. **Generación de código y ejecución.** El plan se traduce a una estructura ejecutable (interpretada como árbol de iteradores en la mayoría de motores). El motor invoca cada iterador en *pipeline*, pidiendo tuplas al storage engine, aplicando filtros y materializando los pasos de agregación. Las tuplas resultantes se envían al cliente.

Donde se conecta cada etapa con las herramientas del DBA.

- Etapas 1-2 fallan con errores SQL clásicos (`syntax error`, `column does not exist`).
- La etapa 4 es la que muestra `EXPLAIN/EXPLAIN ANALYZE`: el plan elegido.
- Para influir en la etapa 4 hay tres palancas: agregar/quitar índices, ejecutar `ANALYZE` (refrescar estadísticas), o usar hints/reescribir la consulta.

Ejemplo

Consulta:

```
SELECT c.nombre, c.apellido, f.monto
FROM cliente c
JOIN factura f ON c.nro_cliente = f.nro_cliente
WHERE f.monto > 100000
ORDER BY f.monto DESC;
```

Paso 1 — Parsing. Tokens `SELECT`, `c.nombre`, `,`, `,`, `...`, $\rightarrow$ árbol sintáctico con nodos *Project*, *Join*, *Filter*, *Sort*.

Paso 2 — Validación. El catálogo confirma que `cliente` y `factura` existen, que tienen las columnas referidas y que el tipo de `monto` permite la comparación con 100000.

Paso 3 — Álgebra relacional (árbol inicial):

$$\Pi_{c.nombre, c.apellido, f.monto}(\sigma_{f.monto > 100000}(cliente \bowtie_{nro_cliente} factura))$$

Paso 4 — Optimización.

- Lógica: el predicado `f.monto > 100000` se empuja a la rama de `factura` con R7a (*push-down*):

$$\Pi_{\dots}(\text{cliente} \bowtie \sigma_{\text{monto} > 100000}(\text{factura}))$$

- Física: si hay índice sobre `factura.monto`, el optimizador puede elegir *Index Range Scan* sobre el predicado; sobre el join, si `cliente.nro_cliente` es PK indexada, suele preferir *Nested Loop* con `factura` ya filtrada como tabla externa.

Paso 5 — Plan de ejecución (ejemplo PostgreSQL):

```
Sort (key=f.monto DESC)
-> Nested Loop
    -> Index Scan using idx_factura_monto on factura f
        Index Cond: (f.monto > 100000)
    -> Index Scan using pk_cliente on cliente c
        Index Cond: (c.nro_cliente = f.nro_cliente)
```

Cada iterador entrega tuplas al de arriba; *Sort* las acumula y emite ordenadas al cliente. Si `factura.monto` tuviese además un índice descendente, *Sort* podría desaparecer.

4. Pregunta 3: Información estadística para estimación de costos

Consigna

¿Qué información estadística se utiliza para estimación de costos de consultas? Dé un ejemplo de estimación de costos.

Idea

El optimizador físico necesita *predecir* qué tan caro saldrá cada plan candidato sin ejecutarlo. Para eso usa estadísticas precalculadas sobre las tablas y sus columnas, almacenadas en el catálogo del motor. Las estadísticas se refrescan con `ANALYZE` (PostgreSQL) o `ANALYZE TABLE` (MySQL); sin estadísticas actualizadas, el optimizador opera a ciegas y puede elegir planes pésimos.

Respuesta

Estadísticas por relación (tabla):

- n_r : cantidad de tuplas (`pg_class.reltuples / INFORMATION_SCHEMA.TABLES.TABLE_ROWS`).
- b_r : cantidad de bloques que ocupa r (`pg_class.relpages / pages` calculados).
- l_r : tamaño promedio de una tupla en bytes.
- f_r : *factor de bloqueo* $= \lfloor B/l_r \rfloor$ donde B es el tamaño de bloque del motor.

Estadísticas por atributo (columna):

- $V(A, r)$: cantidad de valores distintos de A en r (`n_distinct` en `pg_stats`; `CARDINALITY` en `INFORMATION_SCHEMA.STATISTICS`).
- $\min(A, r)$, $\max(A, r)$: rango.
- *Most Common Values (MCVs)* y sus frecuencias (`most_common_vals`, `most_common_freqs` en `pg_stats`): permiten estimar selectividad para igualdades sobre valores populares.
- *Histograma* (equi-depth o equi-width, en `histogram_bounds`): permite estimar selectividad para rangos.
- *Fracción de nulos* (`null_frac`).

- *Correlación* entre orden lógico y físico (**correlation** en pg, no expuesto en MySQL): cercana a 1 indica que el atributo es buen candidato para Index Scan.

Estadísticas por índice (en motores que las usan separadas):

- Altura del árbol h_i .
- Bloques de hoja (**leaf_pages** en `mysql.innodb_index_stats`).
- Cantidad de valores distintos del prefijo del índice.

Fórmulas básicas de estimación:

Cardinalidad de una selección $\sigma_{A=v}(r)$ (selectividad uniforme):

$$\text{tuplas resultantes} \approx \frac{n_r}{V(A, r)}$$

Cardinalidad de un join $r \bowtie_A s$:

$$|r \bowtie s| \approx \frac{n_r n_s}{\max\{V(A, r), V(A, s)\}}$$

(si A es clave en s , la cota se reduce a $|r \bowtie s| \leq n_r$).

Costo de los algoritmos básicos (Teórico 6):

Algoritmo	Operación	Costo
A1 (búsqueda lineal)	σ	$b_r \cdot t_T + t_b$
A3 (igualdad por PK)	$\sigma_{A=v}$, A clave	$(h_i + 1)(t_T + t_b)$
A5 (igualdad por sec.)	$\sigma_{A=v}$	$(h_i + n_r/V(A, r))(t_T + t_b)$
A6 (rango por primario)	$\sigma_{A < v}$	$h_i(t_T + t_b) + b t_T$
A7 (rango por sec.)	$\sigma_{A < v}$	$(h_i + b)(t_T + t_b)$ con b tuplas matcheadas

donde t_T es el tiempo de transferencia de un bloque y t_b el tiempo de búsqueda (*seek*).

Ejemplo

Consideremos la relación **factura** con las siguientes estadísticas:

$$n_r = 100\,000, \quad b_r = 5\,000 \text{ bloques}, \quad V(\text{monto}, r) = 1\,000$$

Costos hipotéticos: $t_T = 1$ ms (transferencia), $t_b = 4$ ms (seek). Índice secundario B+tree sobre **monto** con altura $h_i = 4$.

Consulta: `SELECT * FROM factura WHERE monto = 50000.`

Estimación de cardinalidad: $n_r/V(\text{monto}, r) = 100\,000/1\,000 = 100$ tuplas matchearían.

Costos de los planes alternativos:

Plan A — Búsqueda lineal (A1):

$$\text{Costo}_{A1} = b_r \cdot t_T + t_b = 5\,000 \cdot 1 \text{ ms} + 4 \text{ ms} \approx 5\,004 \text{ ms} \approx 5 \text{ seg}$$

Plan B — Índice secundario (A5):

$$\text{Costo}_{A5} = (h_i + n_r/V(A, r))(t_T + t_b) = (4 + 100)(1 + 4) \text{ ms} = 104 \cdot 5 = 520 \text{ ms}$$

Conclusión. El plan con índice secundario es $\approx 10\times$ más barato que la búsqueda lineal. El optimizador elegirá A5 siempre que la cardinalidad estimada se mantenga baja; si $V(\text{monto}, r)$ fuera mucho más chico (por ejemplo $V = 5$, dando 20 000 matches), el cálculo cambiaría a $\approx 100\,000$ ms con A5 vs los 5 000 ms de A1 y el plan elegido sería el *Seq Scan*.

La curva donde A1 empieza a ser más barato que A5 se llama *break-even point* y depende del cociente entre tuplas matcheadas y total. Para tablas tipo **producto** con pocos valores distintos en una columna (status, tipo, etc.) los índices secundarios suelen no usarse: el motor prefiere leer toda la tabla.

5. Pregunta 4: Algoritmo de Merge Join

Consigna

Explique el algoritmo de Merge Join.

Idea

Merge Join (también llamado *Sort-Merge Join*) es uno de los tres algoritmos físicos clásicos para resolver $r \bowtie_A s$, junto con *Nested Loop* y *Hash Join*. Su idea es la misma que el *merge* del Mergesort: si ambas entradas vienen ordenadas por el atributo de join, basta con recorrerlas una vez en paralelo, comparando los punteros y emitiendo las parejas cuando los valores coinciden. Cuando no vienen ordenadas, el algoritmo puede aún aplicarse pagando un *sort externo* previo en cada entrada.

Respuesta

Precondiciones.

- Las dos relaciones r y s deben estar ordenadas por el atributo (o lista de atributos) de join A .
- Si no lo están, el motor las ordena previamente con un *external merge sort* (típicamente *multi-way merge sort* en disco).
- Si ya hay un índice ordenado sobre A en alguna relación, alcanza con leer el índice secuencialmente para evitar el sort.

Pseudocódigo (caso de join natural por igualdad sobre A):

```

i := 0; j := 0
while i < |r| and j < |s|:
    if r[i].A < s[j].A:
        i := i + 1
    elif r[i].A > s[j].A:
        j := j + 1
    else:
        // r[i].A == s[j].A: match
        v := r[i].A
        // delimitamos el bloque de s con valor v
        ts := j
        while j < |s| and s[j].A == v:
            j := j + 1
        te := j
        // por cada tupla de r con valor v, emitir producto con [ts..te)
        while i < |r| and r[i].A == v:
            for k from ts to te - 1:
                output (r[i], s[k])
            i := i + 1

```


Análisis de costo.

- *Caso óptimo* (ambas relaciones ya ordenadas):

$$\text{Costo} = b_r + b_s \quad (\text{bloques transferidos})$$

Es decir, se lee cada relación una sola vez.

- *Caso con sort previo* (external merge sort, M buffers en memoria):

$$\text{Costo} \approx b_r + b_s + 2b_r \lceil \log_{M-1} \lceil b_r/M \rceil \rceil + 2b_s \lceil \log_{M-1} \lceil b_s/M \rceil \rceil$$

En la práctica, con M suficientemente grande, el sort lleva $\approx 2 \cdot 2 \cdot (b_r + b_s)$ transferencias.

Propiedades y comparación.

- Coste lineal sobre los inputs cuando vienen ordenados $\Rightarrow$ excelente para relaciones grandes con índice o que ya se ordenaron por otro paso del plan.
- *Preserva el orden* de salida sobre A , lo que permite eliminar un `ORDER BY` posterior por el mismo atributo. Hash Join no preserva orden; Nested Loop tampoco.
- Maneja *duplicados* correctamente con el bucle interno que delimita el bloque de tuplas con el mismo valor en s .
- Bueno con *discos rotacionales* porque el acceso es secuencial; con SSD la ventaja respecto de hash join se reduce.
- *No funciona* para condiciones de join no equi-join (rangos, $\neq$, expresiones); en esos casos el motor cae a nested loop o hash join (este último solo soporta equi-joins también).

Decisión del optimizador.

- Si ambas entradas tienen índice ordenado sobre A y no caben en memoria, Merge Join es típicamente el ganador.
- Si una entrada cabe en memoria, Hash Join suele ganar (construye tabla hash sobre la chica y prueba con la grande).
- Si una entrada es muy chica y la otra muy grande con índice sobre A , Nested Loop puede ganar (recorre la chica y hace lookups en la grande).

Ejemplo

Sea r ordenada por A y s ordenada por A :

$r.A$	$r.B$	$s.A$	$s.C$
1	a	1	x
1	b	2	y
3	c	2	z
4	d	4	w

Traza del algoritmo:

1. $i=0, j=0$: $r[0].A=1 = s[0].A=1 \rightarrow$ match. Delimitar bloque de s con valor 1: $[s[0]]$. Por cada r con valor 1: $r[0]$ y $r[1]$. Emite $(1, a, x)$ y $(1, b, x)$. Avanza i a 2, j a 1.
2. $i=2, j=1$: $r[2].A=3$ vs $s[1].A=2$. $3 > 2 \rightarrow$ avanza j a 2.
3. $i=2, j=2$: $r[2].A=3$ vs $s[2].A=2$. $3 > 2 \rightarrow$ avanza j a 3.
4. $i=2, j=3$: $r[2].A=3$ vs $s[3].A=4$. $3 < 4 \rightarrow$ avanza i a 3.
5. $i=3, j=3$: $r[3].A=4 = s[3].A=4 \rightarrow$ match. Emite $(4, d, w)$. Avanza ambos.
6. Fin.

Resultado: $\{(1, a, x), (1, b, x), (4, d, w)\}$.

Cada tupla de r se leyó exactamente una vez (4 lecturas), cada tupla de s una vez (4 lecturas).
Total: $b_r + b_s$ transferencias.

6. Pregunta 5: Diagrama jerárquico Open GIS

Consigna

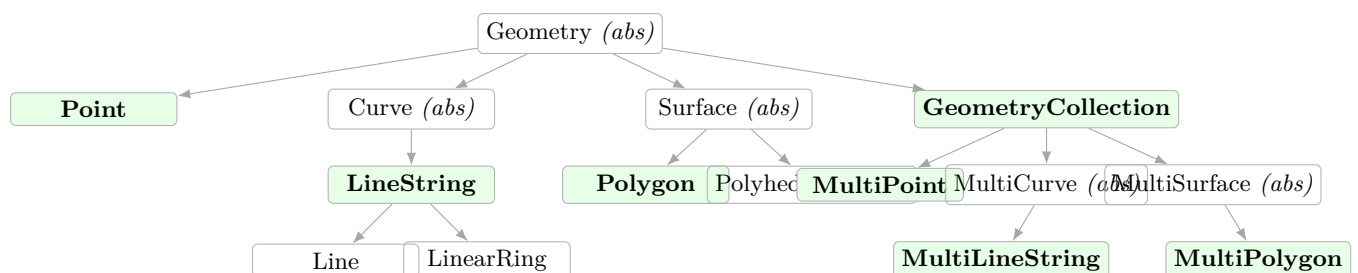
Muestre Diagrama jerárquico tipos de geometrías de Open GIS y ejemplifique gráficamente 3 de tipos de geometrías instanciables.

Idea

El estándar *OGC Simple Features Access* (SFA), publicado por el *Open Geospatial Consortium*, define una jerarquía de tipos para representar entidades geográficas en bases de datos espaciales (PostGIS, Oracle Spatial, MySQL Spatial). La raíz es la clase abstracta **Geometry**; sus descendientes se dividen en *primitivas básicas* (Point, Curve, Surface) y *colecciones* (GeometryCollection y sus subtipos). Solo algunos descendientes son **instanciables**: los demás son clases abstractas que sirven para organizar la jerarquía.

Respuesta

Diagrama jerárquico (las clases en **negrita** son instanciables; el resto son abstractas):



Tipos instanciables (los que efectivamente pueden almacenarse y consultarse):

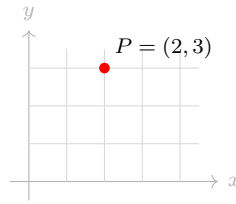
- **Point**: un único punto (x, y) (o (x, y, z) en 3D).
- **LineString**: secuencia de puntos conectados por segmentos rectos.
- **Polygon**: región delimitada por un anillo exterior (LinearRing cerrado) y opcionalmente anillos interiores (*holes*).
- **MultiPoint**, **MultiLineString**, **MultiPolygon**: colecciones homogéneas de los anteriores.
- **GeometryCollection**: colección heterogénea.

Tipos abstractos (existen para tipificar pero no se pueden instanciar): **Geometry**, **Curve**, **Surface**, **MultiCurve**, **MultiSurface**.

Ejemplo

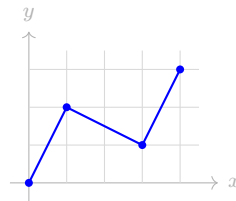
Tres geometrías instanciables, con su WKT (*Well-Known Text*, la representación textual estándar):

1) *Point*. Un único par de coordenadas $(2, 3)$.



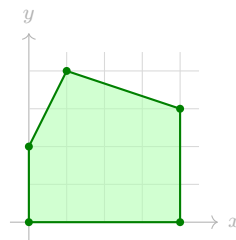
WKT: POINT(2 3)

2) *LineString*. Polilínea por los puntos (0,0), (1,2), (3,1), (4,3).



WKT: LINESTRING(0 0, 1 2, 3 1, 4 3)

3) *Polygon*. Polígono cerrado con anillo exterior (0,0) → (4,0) → (4,3) → (1,4) → (0,2) → (0,0).



WKT: POLYGON((0 0, 4 0, 4 3, 1 4, 0 2, 0 0))

Notar que en POLYGON el primer punto se repite al final del anillo (0 0 aparece dos veces): es el requisito de *LinearRing*, que debe estar cerrado. Si el polígono tuviera agujeros (*holes*), se agregarían como anillos interiores adicionales dentro del paréntesis exterior: POLYGON((exterior...), (hole1...), (hole2...)).

Uso en SQL espacial (PostGIS):

```
CREATE TABLE lugares (
  id    serial PRIMARY KEY,
  geom  geometry(Polygon, 4326)
);

INSERT INTO lugares (geom)
VALUES (ST_GeomFromText('POLYGON((0 0, 4 0, 4 3, 1 4, 0 2, 0 0))', 4326));

SELECT id, ST_Area(geom), ST_AsText(geom) FROM lugares;
```

Cierre

El examen recorre tres bloques temáticos del cuatrimestre:

1. **Concurrencia** (Pregunta 1): marcas temporales como alternativa pesimista a 2PL, libre de deadlocks pero con rollbacks en cascada y starvation.

2. **Procesamiento y optimización** (Preguntas 2–4): el pipeline completo desde el parser hasta el plan físico, las estadísticas que alimentan al optimizador y un algoritmo concreto de join basado en orden.
3. **Modelos no relacionales** (Pregunta 5): la jerarquía OGC de tipos espaciales y la distinción entre clases abstractas y geometrías instanciadas.

La idea común detrás de las cinco preguntas es la misma que atraviesa toda la materia: *el motor toma decisiones por nosotros*, y lo que como usuarios podemos hacer es entender el modelo conceptual (qué garantiza un protocolo, qué fórmulas usa el optimizador, qué tipos define un estándar) para escribir mejor SQL y diseñar mejores esquemas.